

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

ĐẠI HỌC KHOA HỌC TỰ NHIÊN

-----*-----

KHOA ĐIỆN TỬ VIỄN THÔNG

CHUYÊN NGÀNH MÁY TÍNH – HỆ THỐNG NHÚNG

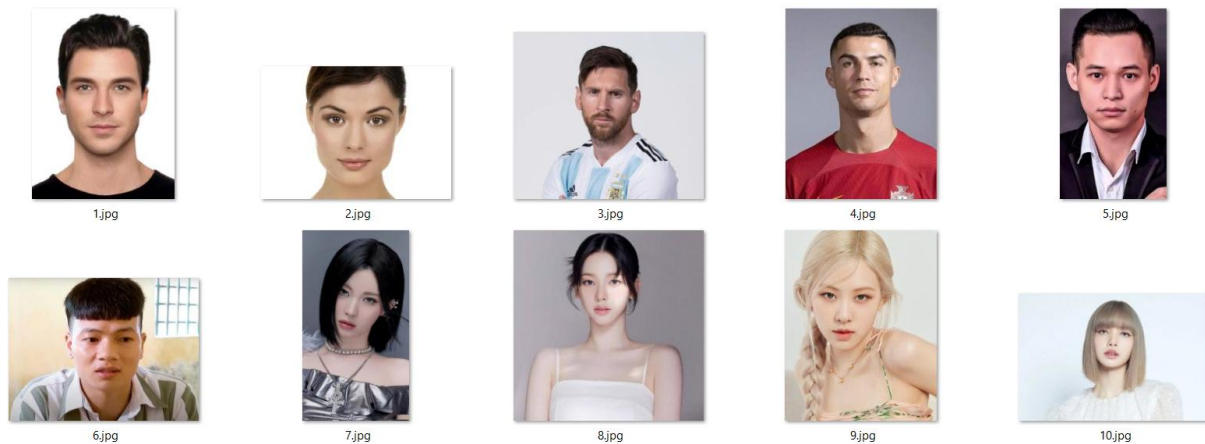
BÀI TẬP 2

Môn: Nhập môn Xử lý ảnh và Video

Họ và Tên	MSSV
Đinh Tiến Dũng	23200008

BÀI TẬP: Lấy 10 khuôn mặt 64x64 -> chuyển sang grayscale -> edge detect-> Tạo 10 vector 64 chiều. So sánh khoảng cách Manhattan.

Github: <https://github.com/DinhTienDung14/image-processing->



Ảnh gốc

1. Chuyển ảnh thông thường thành ảnh grayscale 64x64

- Sử dụng python (file convertRGBraw.py)

```
from PIL import Image

import os

os.makedirs("raw", exist_ok=True)

for i in range(1,11):

    img = Image.open(f"anhgoc/{i}.jpg")
    img = img.convert("RGB")
    img = img.resize((64,64))
    with open(f"RGB_raw/{i}.raw", "wb") as f:
        f.write(img.tobytes())
```

- Chuyển ảnh RGB thành grayscale sử dụng công thức: (file grayscale.c)

$$Gray = 0.299R + 0.587G + 0.114B$$

```
for (int j = 0; j < HEIGHT; j++){
    for (int k = 0; k < WIDTH; k++){
        fread(&R, sizeof(unsigned char), 1, input);
        fread(&G, sizeof(unsigned char), 1, input);
        fread(&B, sizeof(unsigned char), 1, input);
```

```

        gray_value = 0.299* R + 0.587 * G + 0.114 * B;
        gray = (unsigned char)(gray_value + 0.5f);
        fwrite(&gray, sizeof(unsigned char), 1, output);
    }
}

```

- Ảnh trước và sau chuyển:



Ảnh gốc



Ảnh grayscale 64x64

2. Phát hiện 4 cạnh theo hướng

2.1 Các kernel phát triển cạnh (file reponse_raw.c)

- K_H : Horizontol – Hướng ngang

```

int KH[5][5] = {
    { 0, 0, 0, 0, 0},
    { 1, 1, 1, 1, 1},
    { 0, 0, 0, 0, 0},
    {-1, -1, -1, -1, -1},
    { 0, 0, 0, 0, 0}
};

```

- K_P : $+45^\circ$

```

int KP[5][5] = {
    { 0, 0, 0, 1, 0},

```

```

    { 0, 1, 1, 0, -1},
    { 0, 1, 0, -1, 0},
    { 1, 0, -1, -1, 0},
    { 0, -1, 0, 0, 0}
};

```

- K_V : Vertical - Hướng dọc

```

int KV[5][5] = {
    { 0, 1, 0, -1, 0},
    { 0, 1, 0, -1, 0},
    { 0, 1, 0, -1, 0},
    { 0, 1, 0, -1, 0},
    { 0, 1, 0, -1, 0}
};

```

- K_M : - 45°

```

int KM[5][5] = {
    { 0, -1, 0, 0, 0},
    { 1, 0, -1, -1, 0},
    { 0, 1, 0, -1, 0},
    { 0, 1, 1, 0, -1},
    { 0, 0, 0, 1, 0}
};

```

2.2 Tính phản hồi của kernel theo 4 hướng (file reponse_raw.c)

- Tại mỗi pixel, thực hiện phép nhân từng phần tử của kernel với pixel tương ứng trong vùng lân cận 5×5 , sau đó cộng các kết quả để thu được phản hồi theo hướng tương ứng.

$$I_d(x, y) = \left| \sum_{i=-2}^2 \sum_{j=-2}^2 K_d(i, j) \cdot I(x + i, y + j) \right|$$

$$d \in \{H, P, V, M\},$$

```

int ApplyKernel(unsigned char image[HEIGHT][WIDTH], int kernel[5][5], int x, int y) {

```

```

int sum = 0;
for (int i = 0; i < KERNEL_SIZE; i++){
    for (int j = 0; j < KERNEL_SIZE; j++){
        int pixel_x = x + j - 2;
        int pixel_y = y + i - 2;
        sum += kernel[i][j] * image[pixel_y][pixel_x];
    }
}
if (sum < 0) sum = -sum;
return sum;
}

/////////////////////////////////////////////////////////////////

    for (int d = 0; d < NUM_KERNELS; d++){
// Tính phản hồi của kernel trên toàn bộ ảnh
    for (int y = 0; y < HEIGHT; y++){
        for (int x = 0; x < WIDTH; x++){
            if (x < 2 || x >= WIDTH - 2 || y < 2 || y >= HEIGHT - 2) response[d][y][x] = 0;
            else{
                response[d][y][x] = (uint16_t)ApplyKernel(image, kernels[d], x, y);
            }
        }
    }

    snprintf(output_name, sizeof(output_name), "Response_raw/response%d_%s.raw", n,
direction[d]);

    FILE *output = fopen(output_name, "wb");

    if (output == NULL) return 1;

// ghi dữ liệu phản hồi vào file RAW

    count = fwrite(response[d], sizeof(uint16_t), WIDTH * HEIGHT, output);

    fclose(output);

    if (count != WIDTH * HEIGHT) return 1;

}

```

2.3 Tạo 4 bản đồ cạnh (file edge.c)

2.3.a Tính trung vị median và ngưỡng cục bộ

$$TH(x, y) = M_{ed}(x, y) \times 5,$$

```

float CalculateMedian(int values[40]){
    int temp;
    for (int i = 1; i < 40; i++){
        temp = values[i];
        int j = i - 1;
        while (j >= 0 && values[j] > temp){
            values[j + 1] = values[j];
            j--;
        }
        values[j + 1] = temp;
    }
    return (values[19] + values[20]) / 2.0f;
}

float CalculateThreshold(unsigned char image[HEIGHT][WIDTH], int x, int y){
    int differences[40];
    int count = 0;
    int difference;
    for (int n = -2; n <= 1; n++){
        for (int m = -2; m <= 2; m++){
            difference = (int)image[y + m][x + n + 1] - (int)image[y + m][x + n];
            differences[count] = abs(difference);
            count++;
        }
    }
    for (int n = -2; n <= 1; n++){
        for (int m = -2; m <= 2; m++){
            difference = (int)image[y + n + 1][x + m] - (int)image[y + n][x + m];
            differences[count] = abs(difference);
            count++;
        }
    }
    return CalculateMedian(differences) * 5.0f;
}

```

}

2.3.b Chọn hướng có phản hồi lớn nhất

- Tại mỗi pixel có 4 phản hồi

$$I_H(x, y), I_P(x, y), I_V(x, y), I_M(x, y)$$

- Tìm giá trị lớn nhất, hướng có giá trị lớn nhất được đánh dấu bằng 1, các hướng còn lại bằng 0.

//tạo 4 edge map

```
for (int y = 0; y < HEIGHT; y++){
    for (int x = 0; x < WIDTH; x++){
        if (x < 2 || x >= WIDTH - 2 || y < 2 || y >= HEIGHT - 2){
            for (int d = 0; d < NUM_DIRECTIONS; d++){
                edge[d][y][x] = 0;
            }
            continue;
        }
    }
}
```

//Tìm phản hồi lớn nhất

```
uint16_t max_response = response[H][y][x];
```

```
for (int d = 1; d < NUM_DIRECTIONS; d++){
    if (response[d][y][x] > max_response){
        max_response = response[d][y][x];
    }
}
```

```
float threshold = CalculateThreshold(image, x, y); // ngưỡng cục bộ
```

//ngưỡng áp dụng, hướng

```
for (int d = 0; d < NUM_DIRECTIONS; d++){
    if (response[d][y][x] == max_response && response[d][y][x] > threshold){
        edge[d][y][x] = 1;
    }
    else edge[d][y][x] = 0;
}
}
```

```

    }
// Lưu 4 edge maps
    for (int d = 0; d < NUM_DIRECTIONS; d++){
        snprintf(output_name, sizeof(output_name), "Edge_raw/edge%d_%s.raw", image_number,
direction[d]);
        output = fopen(output_name, "wb");
        if (output == NULL) return 1;

        count = fwrite(edge[d], sizeof(unsigned char), WIDTH * HEIGHT, output);
        fclose(output);
        if (count != WIDTH * HEIGHT) return 1;
    }
}

```

- Ảnh edge map theo 4 hướng:



Edge_H



Edge_P



Edge_V



Edge_M

3. Trích xuất vector đặc trưng APED (file createVector64.c)

- Đọc 4 file edge map của từng ảnh, chia mỗi edge map thành 16 vùng, đếm số pixel cạnh của từng vùng rồi ghép thành vector 64 chiều.

```

for (int image_number = 1; image_number <= 10; image_number++){
// đọc 4 map edge
    for (int d = 0; d < NUM_DIRECTIONS; d++){
        snprintf(input_name, sizeof(input_name), "Edge_raw/edge%d_%s.raw", image_number,
direction[d]);
        input = fopen(input_name, "rb");
        if (input == NULL){
            printf("Khong the mo %s\n", input_name);
            fclose(output);
            return 1;
        }
    }
}

```



```

    }

    count = fread(edge[d], sizeof(unsigned char), WIDTH * HEIGHT, input);

    fclose(input);

    if (count != WIDTH * HEIGHT){

        fclose(output);

        return 1;

    }

}

```

// Tính 16 đặc trưng cho mỗi hướng

```

int feature_index = 0;

for (int d = 0; d < NUM_DIRECTIONS; d++){

    for (int a = 0; a < 4; a++){

        for (int b = 0; b < 4; b++){

            int sum = 0;

            for (int y = a * CELL_SIZE; y < (a + 1) * CELL_SIZE; y++){

                for (int x = b * CELL_SIZE; x < (b + 1) * CELL_SIZE; x++){

                    sum += edge[d][y][x];

                }

            }

            feature[feature_index] = sum;

            feature_index++;

        }

    }

}

```

- Vector được ghi vào file APED_features.csv

Image	H_0.0	H_0.1	H_0.2	H_0.3	H_1.0	H_1.1	H_1.2	H_1.3	H_2.0	H_2.1	H_2.2	H_2.3	H_3.0	H_3.1	H_3.2	H_3.3	P_0.0	P_0.1	P_0.2	P_0.3	P_1.0	P_1.1	P_1.2	P_1.3	P_2.0	P_2.1	P_2.2	P_2.3	P_3.0	P_3.1	P_3.2	P_3.3
1	16	118	108	18	17	80	80	5	17	81	83	19	60	95	96	77	10	44	29	17	23	40										
2	8	60	49	4	10	85	85	9	4	52	55	6	3	82	59	0	12	71	33	11	15	34										
3	31	79	86	60	48	66	48	47	28	68	61	27	29	22	44	21	24	51	32	36	26	41										
4	181	103	104	131	148	66	68	66	129	79	78	57	76	72	60	62	22	49	30	4	49	37										
5	50	107	102	76	57	133	132	62	62	110	145	75	46	59	35	65	41	58	5	15	26	36										
6	110	78	29	59	102	61	66	73	104	47	54	64	22	44	55	12	12	42	32	13	22	47										
7	77	95	98	68	76	81	114	68	103	145	140	107	70	65	84	75	87	56	52	29	58	57										
8	9	54	46	41	5	62	34	41	29	103	116	49	4	160	95	30	53	54	35	49	33	37										
9	42	72	36	55	48	116	57	75	43	77	73	139	65	71	110	99	79	64	17	49	42	41										
10	88	92	96	94	86	48	60	86	84	90	38	102	12	53	38	25	91	53	47	61	93	54										

- Mỗi hàng là 1 vector

4. Tính khoảng cách nhỏ nhất và các vị trí tương ứng (file compare.c)

4.1 Khoảng cách Manhattan

- Hai vector đặc trưng X, T có khoảng cách Manhattan được tính theo công thức:

$$d(X, T) = \sum_{k=1}^{64} |X_k - T_k|$$

```
int ManhattanDistance(int X[FEATURE_SIZE],int T[FEATURE_SIZE]){
    int distance = 0;
    for (int i = 0; i < FEATURE_SIZE; i++){
        distance += abs(X[i] - T[i]);
    }
    return distance;
}
```

- Để tìm khoảng cách Manhattan nhỏ nhất, ta đi so sánh từng cặp ảnh với nhau theo công thức:

$$d(i, j) = \sum_{k=0}^{63} |X_{i,k} - X_{j,k}|$$

- Khoảng cách của cặp ảnh

```
for (int i = 0; i < NUM_IMAGES; i++){
    for (int j = 0; j < NUM_IMAGES; j++){
        if (i == j){
            distance[i][j] = 0;
        }
        else{
            distance[i][j] = ManhattanDistance(features[i], features[j]);
        }
    }
}
```

- Khoảng cách của các ảnh với nhau

```
printf("\nManhattan Distance Matrix\n\n");
printf("%8s", "");
for (int j = 0; j < NUM_IMAGES; j++){
    printf("%8d", image_id[j]);
}
printf("\n");
for (int i = 0; i < NUM_IMAGES; i++){
```

```

printf("%8d", image_id[i]);

for (int j = 0; j < NUM_IMAGES; j++){
    printf("%8d", distance[i][j]);
}

printf("\n");
}

```

- Tìm ảnh gần nhất với mỗi ảnh

```

for (int i = 0; i < NUM_IMAGES; i++){
    int min_distance = -1;
    int nearest_index = -1;

    for (int j = 0; j < NUM_IMAGES; j++){
        if (i == j) continue;
        if (min_distance == -1 || distance[i][j] < min_distance){
            min_distance = distance[i][j];
            nearest_index = j;
        }
    }
}
}

```

- Kết quả:

```

Manhattan Distance Matrix

```

	1	2	3	4	5	6	7	8	9	10
1	0	1304	1780	1853	1743	1962	1596	2018	1786	2321
2	1304	0	2070	2367	2505	1944	2368	2222	2246	2545
3	1780	2070	0	1769	1931	1372	1944	1690	1644	1675
4	1853	2367	1769	0	2050	1777	1669	2375	1959	1832
5	1743	2505	1931	2050	0	2141	1523	2245	1689	1994
6	1962	1944	1372	1777	2141	0	1992	1914	1788	1529
7	1596	2368	1944	1669	1523	1992	0	2154	1588	1801
8	2018	2222	1690	2375	2245	1914	2154	0	1778	1947
9	1786	2246	1644	1959	1689	1788	1588	1778	0	1827
10	2321	2545	1675	1832	1994	1529	1801	1947	1827	0

```

Nearest face image for each image
Image 1 -> Image 2, distance = 1304
Image 2 -> Image 1, distance = 1304
Image 3 -> Image 6, distance = 1372
Image 4 -> Image 7, distance = 1669
Image 5 -> Image 7, distance = 1523
Image 6 -> Image 3, distance = 1372
Image 7 -> Image 5, distance = 1523
Image 8 -> Image 3, distance = 1690
Image 9 -> Image 7, distance = 1588
Image 10 -> Image 6, distance = 1529

```

FILE CODE

convertRGBraw.py

```
from PIL import Image

import os

os.makedirs("raw", exist_ok=True)

for i in range(1,11):

    img = Image.open(f"anhgoc/{i}.jpg")
    img = img.convert("RGB")
    img = img.resize((64,64))
    with open(f"RGB_raw/{i}.raw", "wb") as f:
        f.write(img.tobytes())
```

grayscale.c

```
#include <stdio.h>
#include <stdint.h>

#define WIDTH 64
#define HEIGHT 64

int main()
{
    FILE *input;
    FILE *output;
    unsigned char R;
    unsigned char G;
    unsigned char B;
    unsigned char gray;
    float gray_value;
    char input_name[32];
    char output_name[32];

    for (int i = 1; i <= 10; i++){
```

```

    snprintf(input_name, sizeof(input_name), "RGB_raw/%d.raw", i);
    snprintf(output_name, sizeof(output_name), "Gray_raw/gray%d.raw", i);
    input = fopen(input_name, "rb");
    if (input == NULL) return 1;
    output = fopen(output_name, "wb");
    if (output == NULL)
    {
        fclose(input);
        return 1;
    }
    for (int j = 0; j < HEIGHT; j++){
        for (int k = 0; k < WIDTH; k++){
            fread(&R, sizeof(unsigned char), 1, input);
            fread(&G, sizeof(unsigned char), 1, input);
            fread(&B, sizeof(unsigned char), 1, input);
            gray_value = 0.299* R + 0.587 * G + 0.114 * B;
            gray = (unsigned char)(gray_value + 0.5f);
            fwrite(&gray, sizeof(unsigned char), 1, output);
        }
    }
    fclose(input);
    fclose(output);
}
return 0;
}

```

response_raw.c

```

#include <stdio.h>
#include <stdint.h>
#define WIDTH 64
#define HEIGHT 64
#define KERNEL_SIZE 5
#define NUM_KERNELS 4

```

```
int KH[5][5] = {
    { 0, 0, 0, 0, 0},
    { 1, 1, 1, 1, 1},
    { 0, 0, 0, 0, 0},
    {-1, -1, -1, -1, -1},
    { 0, 0, 0, 0, 0}
};

int KP[5][5] = {
    { 0, 0, 0, 1, 0},
    { 0, 1, 1, 0, -1},
    { 0, 1, 0, -1, 0},
    { 1, 0, -1, -1, 0},
    { 0, -1, 0, 0, 0}
};

int KV[5][5] = {
    { 0, 1, 0, -1, 0},
    { 0, 1, 0, -1, 0},
    { 0, 1, 0, -1, 0},
    { 0, 1, 0, -1, 0},
    { 0, 1, 0, -1, 0}
};

int KM[5][5] = {
    { 0, -1, 0, 0, 0},
    { 1, 0, -1, -1, 0},
    { 0, 1, 0, -1, 0},
    { 0, 1, 1, 0, -1},
    { 0, 0, 0, 1, 0}
};

int ApplyKernel(unsigned char image[HEIGHT][WIDTH],int kernel[5][5],int x,int y){
    int sum = 0;
    for (int i = 0; i < KERNEL_SIZE; i++){
        for (int j = 0; j < KERNEL_SIZE; j++){
            int pixel_x = x + j - 2;
            int pixel_y = y + i - 2;
```

```

        sum += kernel[i][j] * image[pixel_y][pixel_x];
    }
}
if (sum < 0) sum = -sum;
return sum;
}
int main(){
    unsigned char image[HEIGHT][WIDTH];
    uint16_t response[NUM_KERNELS][HEIGHT][WIDTH];
    int (*kernels[NUM_KERNELS])[5] = { KH, KP, KV, KM };
    char input_name[32];
    char output_name[64];
    char *direction[NUM_KERNELS] = {"H", "P", "V", "M"};
    for (int n = 1; n <= 10; n++)
    {
        snprintf(input_name, sizeof(input_name), "Gray_raw/gray%d.raw", n);
        FILE *input = fopen(input_name, "rb");
        if (input == NULL) return 1;
        size_t count = fread( image, sizeof(unsigned char), WIDTH * HEIGHT, input);
        fclose(input);
        if (count != WIDTH * HEIGHT) return 1;

        for (int d = 0; d < NUM_KERNELS; d++){
            for (int y = 0; y < HEIGHT; y++){
                for (int x = 0; x < WIDTH; x++){
                    if (x < 2 || x >= WIDTH - 2 || y < 2 || y >= HEIGHT - 2) response[d][y][x] = 0;
                    else{
                        response[d][y][x] = (uint16_t)ApplyKernel(image, kernels[d], x, y);
                    }
                }
            }
        }

        snprintf(output_name, sizeof(output_name), "Response_raw/response%d_%s.raw", n,
direction[d]);
        FILE *output = fopen(output_name, "wb");

```

```

        if (output == NULL) return 1;

        count = fwrite(response[d], sizeof(uint16_t), WIDTH * HEIGHT, output);

        fclose(output);

        if (count != WIDTH * HEIGHT) return 1;
    }
}

return 0;
}

```

edge.c

```

#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#define WIDTH 64
#define HEIGHT 64
#define NUM_DIRECTIONS 4
#define H 0
#define P 1
#define V 2
#define M 3
float CalculateMedian(int values[40]){
    int temp;
    for (int i = 1; i < 40; i++){
        temp = values[i];
        int j = i - 1;
        while (j >= 0 && values[j] > temp){
            values[j + 1] = values[j];
            j--;
        }
        values[j + 1] = temp;
    }
    return (values[19] + values[20]) / 2.0f;
}

float CalculateThreshold(unsigned char image[HEIGHT][WIDTH], int x, int y){

```



```

int differences[40];

int count = 0;

int difference;

for (int n = -2; n <= 1; n++){
    for (int m = -2; m <= 2; m++){
        difference = (int)image[y + m][x + n + 1] - (int)image[y + m][x + n];
        differences[count] = abs(difference);
        count++;
    }
}

for (int n = -2; n <= 1; n++){
    for (int m = -2; m <= 2; m++){
        difference = (int)image[y + n + 1][x + m] - (int)image[y + n][x + m];
        differences[count] = abs(difference);
        count++;
    }
}

return CalculateMedian(differences) * 5.0f; //ngưỡng
}

int main()
{
    unsigned char image[HEIGHT][WIDTH];
    uint16_t response[NUM_DIRECTIONS][HEIGHT][WIDTH];
    unsigned char edge[NUM_DIRECTIONS][HEIGHT][WIDTH];
    char input_name[64];
    char output_name[64];
    char *direction[NUM_DIRECTIONS] = {"H", "P", "V", "M"};
    FILE *input;
    FILE *output;
    size_t count;

    for (int image_number = 1; image_number <= 10; image_number++){
        snprintf(input_name, sizeof(input_name), "Gray_raw/gray%d.raw", image_number);
        input = fopen(input_name, "rb");
        if (input == NULL) return 1;

```

```

count = fread(image, sizeof(unsigned char), WIDTH * HEIGHT, input);
fclose(input);
if (count != WIDTH * HEIGHT) return 1;
for (int d = 0; d < NUM_DIRECTIONS; d++){
    snprintf(input_name, sizeof(input_name), "Response_raw/response%d_%s.raw",
image_number, direction[d]);
    input = fopen(input_name, "rb");
    if (input == NULL) return 1;
    count = fread(response[d], sizeof(uint16_t), WIDTH * HEIGHT, input);
    fclose(input);
    if (count != WIDTH * HEIGHT) return 1;
}
for (int y = 0; y < HEIGHT; y++){
    for (int x = 0; x < WIDTH; x++){
        if (x < 2 || x >= WIDTH - 2 || y < 2 || y >= HEIGHT - 2){
            for (int d = 0; d < NUM_DIRECTIONS; d++){
                edge[d][y][x] = 0;
            }
            continue;
        }
        uint16_t max_response = response[H][y][x];
        for (int d = 1; d < NUM_DIRECTIONS; d++){
            if (response[d][y][x] > max_response){
                max_response = response[d][y][x];
            }
        }
        float threshold = CalculateThreshold(image, x, y); // ngưỡng cục bộ
        for (int d = 0; d < NUM_DIRECTIONS; d++){
            if (response[d][y][x] == max_response && response[d][y][x] > threshold){
                edge[d][y][x] = 1;
            }
            else edge[d][y][x] = 0;
        }
    }
}

```

```

    }

    for (int d = 0; d < NUM_DIRECTIONS; d++){
        snprintf(output_name, sizeof(output_name), "Edge_raw/edge%d_%s.raw", image_number,
direction[d]);

        output = fopen(output_name, "wb");
        if (output == NULL) return 1;

        count = fwrite(edge[d], sizeof(unsigned char), WIDTH * HEIGHT, output);

        fclose(output);

        if (count != WIDTH * HEIGHT) return 1;
    }
}

return 0;
}

```

createVector64.c

```

#include <stdio.h>

#define WIDTH 64
#define HEIGHT 64
#define CELL_SIZE 16
#define NUM_CELLS 16
#define NUM_DIRECTIONS 4
#define FEATURE_SIZE 64
#define H 0
#define P 1
#define V 2
#define M 3

int main()
{
    unsigned char edge[NUM_DIRECTIONS][HEIGHT][WIDTH];
    int feature[FEATURE_SIZE];

    char input_name[64];

    char *direction[NUM_DIRECTIONS] = {"H", "P", "V", "M"};

    FILE *input;

    FILE *output;

    size_t count;

```

```

output = fopen("APED_features.csv", "w");
if (output == NULL) return 1;
fprintf(output, "Image");
for (int d = 0; d < NUM_DIRECTIONS; d++){
    for (int a = 0; a < 4; a++){
        for (int b = 0; b < 4; b++){
            fprintf(output, ",%s_%d_%d", direction[d], a, b);
        }
    }
}
fprintf(output, "\n");
for (int image_number = 1; image_number <= 10; image_number++)
{
    for (int d = 0; d < NUM_DIRECTIONS; d++){
        snprintf(input_name, sizeof(input_name), "Edge_raw/edge%d_%s.raw", image_number,
direction[d]);
        input = fopen(input_name, "rb");
        if (input == NULL){
            printf("Khong the mo %s\n", input_name);
            fclose(output);
            return 1;
        }
        count = fread(edge[d], sizeof(unsigned char), WIDTH * HEIGHT, input);
        fclose(input);
        if (count != WIDTH * HEIGHT){
            fclose(output);
            return 1;
        }
    }
}
int feature_index = 0;
for (int d = 0; d < NUM_DIRECTIONS; d++){
    for (int a = 0; a < 4; a++){
        for (int b = 0; b < 4; b++){
            int sum = 0;

```

```

        for (int y = a * CELL_SIZE; y < (a + 1) * CELL_SIZE; y++){
            for (int x = b * CELL_SIZE; x < (b + 1) * CELL_SIZE; x++){
                sum += edge[d][y][x];
            }
        }
        feature[feature_index] = sum;
        feature_index++;
    }
}

fprintf(output, "%d", image_number);
for (int i = 0; i < FEATURE_SIZE; i++){
    fprintf(output, ",%d", feature[i]);
}
fprintf(output, "\n");
}
fclose(output);
return 0;
}

```

compare.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define NUM_IMAGES 10
#define FEATURE_SIZE 6
int features[NUM_IMAGES][FEATURE_SIZE];
int image_id[NUM_IMAGES];
int ManhattanDistance(int X[FEATURE_SIZE],int T[FEATURE_SIZE]){
    int distance = 0;
    for (int i = 0; i < FEATURE_SIZE; i++){
        distance += abs(X[i] - T[i]);
    }
    return distance;
}

```

```

}
int ReadFeatures(const char *filename)
{
    FILE *input = fopen(filename, "r");
    if (input == NULL){
        printf("Cannot open %s\n", filename);
        return 0;
    }
    char line[2048];
    if (fgets(line, sizeof(line), input) == NULL)
    {
        fclose(input);
        return 0;
    }
    int row = 0;
    while (row < NUM_IMAGES && fgets(line, sizeof(line), input) != NULL){
        char *token;
        token = strtok(line, ",\r\n");
        if (token == NULL){
            fclose(input);
            return 0;
        }
        image_id[row] = atoi(token);
        for (int i = 0; i < FEATURE_SIZE; i++){
            token = strtok(NULL, ",\r\n");
            if (token == NULL){
                printf("Invalid feature data at row %d\n", row + 1);
                fclose(input);
                return 0;
            }
            features[row][i] = atoi(token);
        }
        row++;
    }
}

```

```
fclose(input);
if (row != NUM_IMAGES){
    printf("Expected %d images, but read %d\n",NUM_IMAGES, row);
    return 0;
}
return 1;
}
int main()
{
    int distance[NUM_IMAGES][NUM_IMAGES];
    if (!ReadFeatures("APED_features.csv"))
    {
        printf("Failed to read feature vectors.\n");
        return 1;
    }
    for (int i = 0; i < NUM_IMAGES; i++){
        for (int j = 0; j < NUM_IMAGES; j++){
            if (i == j){
                distance[i][j] = 0;
            }
            else{
                distance[i][j] = ManhattanDistance(features[i], features[j]);
            }
        }
    }
    printf("\nManhattan Distance Matrix\n\n");
    printf("%8s", "");
    for (int j = 0; j < NUM_IMAGES; j++){
        printf("%8d", image_id[j]);
    }
    printf("\n");
    for (int i = 0; i < NUM_IMAGES; i++){
        printf("%8d", image_id[i]);
        for (int j = 0; j < NUM_IMAGES; j++){
```

```

        printf("%8d", distance[i][j]);
    }
    printf("\n");
}
printf("\nNearest face image for each image\n\n");
for (int i = 0; i < NUM_IMAGES; i++){
    int min_distance = -1;
    int nearest_index = -1;
    for (int j = 0; j < NUM_IMAGES; j++){
        if (i == j) continue;
        if (min_distance == -1 || distance[i][j] < min_distance){
            min_distance = distance[i][j];
            nearest_index = j;
        }
    }
    printf("Image %d -> Image %d, distance = %d\n", image_id[i], image_id[nearest_index], min_distance);
}
return 0;
}

```